

SIMD architecture overview

I'll walk through this step-by-step and tie all the pieces together, keeping the structure close to how the text flows, but with more intuition and concrete explanations.

1. Big picture: What is SIMD and why do we care?

The chapter starts with a simple but important question:

For "single instruction, multiple data" (SIMD), how many *real* applications actually have enough data-level parallelism (DLP) to make SIMD worth it?

Data-Level Parallelism (DLP) means:

You have one operation that needs to be applied to many independent data items.

Examples:

- Scientific computing: matrix operations, vector operations.
- Media: image processing, audio/video filters, codecs.

In all these, you often do the same arithmetic operation on a large array of numbers. That's ideal for SIMD.

Why SIMD is attractive

1. Energy efficiency vs MIMD

- SIMD: One instruction says "do this operation on all these elements".
- MIMD (multiple instruction, multiple data): You need one instruction per operation, per data item, per core.
- Fewer instructions fetched and decoded $\Rightarrow$ less energy and simpler control.

2. Programmer thinks sequentially

- You write code *as if* you're operating on one element.
- The hardware applies that operation to many elements in parallel.
- This is easier than explicitly dealing with threads and synchronization as in MIMD.

3. Three main SIMD variants covered in the chapter

- **Vector architectures** (classic supercomputers, and now also in CPUs):
Operate on long vectors stored in *vector registers*. Instructions like **ADDV** operate on entire vectors.
- **Multimedia SIMD extensions** (MMX, SSE, AVX on x86):
These are "packed" operations inside ordinary registers. Smaller width than classic vectors but similar idea: do several operations in parallel within one instruction.

- **GPUs (graphics processing units):**

They started for graphics, but now do general purpose computing (GPGPU). They are SIMD/SIMT-like, but in a “heterogeneous” system: CPU + system memory + GPU + GPU memory.

The book makes a prediction using a graph (Figure 4.1):

- Cores (MIMD) roughly double every two years.
- SIMD width (number of operations per cycle) doubles every four years.
- Combined, SIMD parallelism can potentially give **about 2× more speedup** than just adding more cores (MIMD) over the next decade.
- If your application has both:
 - **Thread-Level Parallelism (TLP)** = more cores, and
 - **Data-Level Parallelism (DLP)** = wider SIMD
 you can get almost an **order of magnitude** speedup by 2020 vs early 2000s baseline.

So the chapter’s goal:

Explain **vector architectures** in detail (because they’re the most general form of SIMD), and use that understanding to compare:

- Vector vs multimedia SIMD,
- Vector vs GPU architectures.

2. Vector architectures – core idea

A **vector processor** is designed to work on entire vectors (arrays) of data with single instructions:

- It **gathers** many scattered data elements from memory into large, sequential **vector registers**.
- It performs arithmetic operations on these vector registers.
- It **scatters** the results back to memory.

A single vector instruction → dozens of element-wise operations on independent data elements.

Benefits:

1. Hide memory latency

- Loads/stores for vectors are **deeply pipelined**.
- You pay the long memory latency only once per whole vector, not once per element.

2. Keep memory busy

- Vector loads/stores stream data at high bandwidth (ideally one word per cycle after startup).

3. Compiler-controlled buffering

- Vector registers act as big, explicit buffers that the compiler uses instead of relying entirely on caches.

3. The example architecture: VMIPS

The book introduces a toy vector architecture called **VMIPS**:

- Scalar part: standard MIPS ISA.
- Vector part: a logical vector extension of MIPS.

Main components of VMIPS

1. Vector registers

- 8 vector registers: **V0, V1, ..., V7** (implicitly).
- Each register holds **64 elements**, each **64 bits** (double-precision).
- So each vector register = 64×64 bits = 4096 bits.
- Needs a lot of read/write ports to feed multiple vector functional units in parallel.
Example: ≥ 16 read ports and 8 write ports in total, connected via crossbar switches.

2. Vector functional units

- Fully pipelined arithmetic units (e.g. add, multiply, divide, integer, logical).
- Each unit can start a new operation every clock cycle (initiation interval = 1).
- A control unit detects structural hazards (same unit needed twice) and data hazards (dependencies).

3. Vector load/store unit

- Handles vector loads and stores, ideally with bandwidth 1 word per clock after startup.
- Also typically handles scalar loads/stores.

4. Scalar registers

- Ordinary MIPS: 32 general-purpose integer registers, 32 FP registers.
- Scalar registers can:
 - Provide input to vector functional units (for vector-scalar operations),
 - Compute addresses for vector loads/stores.

5. Special registers

- **VLR (Vector Length Register)** – controls how many elements of a vector instruction are active.
- **VM (Vector Mask Register)** – one bit per element, used for conditional execution.

4. VMIPS vector instructions – what can it do?

The instruction naming convention:

- Scalar MIPS: **ADD.D F0, F1, F2**
- Vector VMIPS: append **VV** or **VS**:
 - **ADDVV.D V1, V2, V3** $\rightarrow$ element-wise $V1[i] = V2[i] + V3[i]$

- **ADDVS.D V1, V2, F0** $\rightarrow V1[i] = V2[i] + F0$ (scalar broadcast).

Broadly, there are several categories:

1. Vector arithmetic

- **ADDVV.D, ADDVS.D**
- **SUBVV.D, SUBVS.D, SUBSV.D**
- **MULVV.D, MULVS.D**
- **DIVVV.D, DIVVS.D, DIVSV.D**

Each operates on all active vector elements in a single instruction.

2. Vector loads and stores (contiguous)

- **LV V1, R1** $\rightarrow$ load an entire vector from memory starting at address in **R1**.
- **SV R1, V1** $\rightarrow$ store an entire vector to memory starting at **R1**.

3. Vector loads/stores with stride

- **LVWS V1, (R1, R2)** $\rightarrow$ load with stride (address: **R1 + i * R2**).
- **SVWS (R1, R2), V1** $\rightarrow$ store with stride.

Used for multidimensional arrays where elements in a vector are not consecutive in memory.

4. Gather/Scatter (indexed loads/stores)

- **LVI V1, (R1 + V2)** $\rightarrow$ addresses come from **R1 + V2[i]** (gather).
 - **SVI (R1 + V2), V1** $\rightarrow$ store using addresses **R1 + V2[i]** (scatter).
- Used for sparse data structures where only some indices are relevant.

5. Index vector creation

- **CVI V1, R1** $\rightarrow$ create a vector of indices:
 - **V1 = [0, 1×R1, 2×R1, ..., 63×R1]**

6. Comparison and masks

- **SxxVV.D V1, V2** and **SxxVS.D V1, F0** (xx = EQ, NE, GT, LT, GE, LE)
Compare elements and set **vector mask** bits accordingly.
- **POP R1, VM** $\rightarrow$ population count: count "1" bits in VM.
- **CVM** $\rightarrow$ set VM = all 1s (no masking; all elements active).

7. Moving data to/from special registers

- **MTC1 VLR, R1 / MFC1 R1, VLR** $\rightarrow$ moves between scalar register and vector length register.
- **MVTM VM, F0 / MVFM F0, VM** $\rightarrow$ move between scalar FP register and mask.

5. Why vector instructions are powerful (vs scalar loop)

Classic example: **DAXPY / SAXPY**

Operation: **$Y = a * X + Y$**

- **X** and **Y** vectors in memory, **a** scalar.

Scalar MIPS loop (simplified idea)

Scalar version conceptually does:

```
c
Loop:
  load X[i]
  multiply by a
  load Y[i]
  add
  store back to Y[i]
  i++
  branch if i < n
```

For 64 elements, this becomes:

- $\sim 64 \times (\text{load, mul, load, add, store, increments, branch})$
- So nearly **hundreds of dynamic instructions** executed.

VMIPS vector version

The vector version conceptually:

1. Load scalar **a** into **F0**.
2. **LV V1, Rx** → load entire vector X.
3. **MULVS.D V2, V1, F0** → multiply all $X[i]$ by **a**.
4. **LV V3, Ry** → load entire vector Y.
5. **ADDVV.D V4, V2, V3** → add elements.
6. **SV V4, Ry** → store the result back.

Total: **6 vector instructions** for 64 elements.

Key differences:

1. **Huge reduction in dynamic instruction count**
 ~ 600 scalar instructions → 6 vector instructions.
2. **Pipeline stalls**
 - Scalar pipeline stalls often at each data dependency (e.g., **ADD** waits for **MUL**, **STORE** waits for **ADD**, etc.).
 - In vector, each pipeline stall happens only once per **vector instruction**, not once per element. After startup, elements stream through the pipeline.

That's why vector code is said to be *vectorized*: loops without loop-carried dependencies can be turned into these dense vector sequences.

6. Vector performance model: convoys and chimes

To reason about performance, the book defines:

1. **Initiation rate:**

How many new elements per clock a vector unit can accept.

In simple VMIPS: one element per clock per unit.

2. **Convoy:**

A set of vector instructions that can **run together** (overlapped) because:

- No structural hazards (not fighting for same functional unit in same cycle),
- Dependencies can be satisfied with chaining (forwarding between pipes).

3. **Chime:**

- Time to execute **one convoy**.
- If you have m convoys and vector length n , total time $\approx m \times n$ cycles (for single-lane VMIPS) ignoring startup.

Chaining

Chaining = vector form of forwarding:

- A consumer vector instruction can start using the first produced elements from the producer even before the producer completes the whole vector.
- Example: `LV V1 → MULVS.D V2, V1, F0`
 - `MULVS` can start as soon as the first elements of `V1` are loaded, not after the entire vector is loaded.

Example convoy layout (from your code)

Vector sequence:

1. `LV V1, Rx` ; load X
2. `MULVS.D V2, V1, F0` ; multiply
3. `LV V3, Ry` ; load Y
4. `ADDVV.D V4, V2, V3` ; add
5. `SV V4, Ry` ; store

Convoy grouping:

- **Convoy 1:** `LV V1, Rx` and `MULVS.D V2, V1, F0`
 - `MULVS` depends on `LV` but can chain.
- **Convoy 2:** `LV V3, Ry` and `ADDVV.D V4, V2, V3`
 - Again, `ADD` chains from `LV`.
- **Convoy 3:** `SV V4, Ry`
 - Structural hazard with `LV` in convoy 2 (load/store unit), so it must be alone.

So:

- $\# \text{convoys} = 3 \rightarrow 3 \text{ chimes}$.

- Each chime $\approx$ **vector length** cycles (assuming 1 element per cycle).
- For vector length 64:
 - $\approx 3 \times 64 = 192$ cycles total for the arithmetic part.
- There are 2 FLOPs per element (mul + add), so total FLOPs = $2 \times 64 = 128$.
Roughly 192 cycles / 128 FLOPs $\approx$ **1.5 cycles/FLOP** (ignoring issue+startup).

The **chime model** ignores some overheads like:

- Limits on how many vector instructions you can issue per cycle.
- Start-up latency of pipelines (especially loads).

But for *long* vectors, these overheads are relatively small compared to n .

7. Multiple lanes: how to get >1 element per cycle

So far, VMIPS is assumed to have **one lane** per vector unit:

- 1 element per cycle per vector unit.

To increase throughput:

- Use **multiple lanes**: e.g., 4 lanes $\rightarrow$ 4 elements per cycle.

Structure (conceptually like Figure 4.5):

- **Vector register file is split** into multiple slices (one per lane):
 - Lane 0: elements 0, 4, 8, ...
 - Lane 1: elements 1, 5, 9, ...
 - Lane 2: elements 2, 6, 10, ...
 - Lane 3: elements 3, 7, 11, ...
- Each lane has its own:
 - Portion of the vector registers,
 - Add, multiply, load/store pipelines.

When a vector instruction is executed:

- Lane 0 operates on its elements, lane 1 on its elements, etc.
- The same instruction controls all lanes in lockstep.

Effect:

- Time for a chime drops from 64 cycles (1 lane) to $64/4 = 16$ cycles (4 lanes).

Advantages of multiple lanes:

- No extra complexity in the ISA; only microarchitecture changes.

- Can trade off frequency vs lanes (half frequency, double lanes to keep peak throughput).
- Good scaling: as long as your vectors are long enough that you don't run out of vector instructions to execute.

8. Vector length register (VLR) and strip mining

Real programs *almost never* have vectors of exactly 64 elements:

- The real vector length n might be:
 - Not known at compile time,
 - Larger or smaller than 64,
 - Passed as a function parameter,
 - Different for each call.

Maximum Vector Length (MVL)

- Architecture defines **MVL** (for VMIPS, $MVL = 64$).
- Vector registers physically have MVL elements.
- Actual number of active elements is set by **VLR** ($\leq MVL$).

Case 1: $n \leq MVL$

- Just set **VLR** = n .
- All vector instructions operate on only the first n elements.

Case 2: $n > MVL$ – Strip mining

You must process n elements in **chunks** of size $\leq MVL$. This is **strip mining**:

High-level idea:

1. Split the big vector of length n into segments:
 - First segment: $n \% MVL$ elements (remainder part), length m .
 - Following segments: each of length **MVL**.
2. For each chunk:
 - Set **VLR** to the chunk length (first chunk: m , later chunks: **MVL**).
 - Perform the vector operations on that subset.

Visually (like Figure 4.6):

- Think of indices i from 0 to $n-1$.
- You process:
 - $i = 0$ to $m-1$
 - $i = m$ to $m+MVL-1$

- $i = m + \text{MVL}$ to $m + 2 \times \text{MVL} - 1$
- ... until you reach n .

The C pseudo-code the book gives:

- Outer loop: runs number of chunks ($n / \text{MVL} + \text{remainder}$).
- Inner loop: vectorizable part (with VLR controlling length).

Compilers typically implement this automatically when vectorizing loops with $n > \text{MVL}$.

9. Vector mask register (VM) and IF statements

Loops with conditionals are tricky:

```
c
for (i = 0; i < 64; i++) {
    if (X[i] != 0)
        X[i] = X[i] - Y[i];
}
```

Scalar execution: simple branch per element.

Vector execution: we need a way to say:

- “For these indices, execute; for those indices, skip.”

Vector mask

- **VM** is a boolean vector: one bit per element.
- When executing a masked vector instruction:
 - Elements with $\text{VM}[i] = 1$ are executed.
 - Elements with $\text{VM}[i] = 0$ are **left unchanged** in the destination vector.

To implement the example loop in VMIPS:

1. Load vectors X and Y into vector registers.
2. Compare X to zero to set the mask:
 - Something like: **SNEVS.D V1, F0** ($\text{VM}[i] = 1$ if $V1[i] \neq F0$).
3. Execute **SUBVV.D** under the mask:
 - Only elements where $X[i] \neq 0$ get updated.
4. Store X back.

This transformation from branch-based code to mask-based code is called **if-conversion**.

Trade-off:

- Even when $\text{VM}[i] = 0$, the instruction still takes time (the pipeline still runs).
- But you avoid branch mispredictions, divergence, control hazards, etc.

- For many workloads, masked vector execution is still much faster than falling back to scalar code.

Note: In **vector processors**, VM is explicit in architectural state.

In **GPUs**, an equivalent mask exists internally but is hidden from the programmer; the hardware manages it based on control flow (SIMT model).

10. Memory system: banks and bandwidth

Vector loads/stores have:

- **Start-up time**: latency to get first word into the register.
- **Initiation rate**: ideally 1 word per cycle thereafter.

To maintain throughput:

- Memory system must supply enough bandwidth.
- Real systems use **many independent memory banks** so multiple addresses can be serviced in parallel.

Example (from the text):

- Cray T932:
 - 32 processors.
 - Each processor can issue 4 loads + 2 stores per cycle → 6 references/processor/cycle.
 - Total = $32 \times 6 = 192$ memory refs per cycle.
 - Memory bank cycle time = 15 ns, CPU cycle = ~ 2.167 ns $\Rightarrow$ each bank busy for ~ 7 CPU cycles.
 - To avoid stalls, you'd need $\approx 192 \times 7 = 1344$ banks!

In reality, they had 1024 banks and later improved memory technology to reduce cycle time and increase effective bandwidth.

The key idea:

- Vector load/store units are like powerful, predictable **prefetch engines**.
- They feed data in streams, and the memory system with many banks must keep up.

11. Stride: handling multidimensional arrays

For multidimensional arrays (e.g. matrices), elements we want in a vector may not be contiguous in memory.

Example: matrix multiply:

```
c
for (i = 0; i < 100; i++)
  for (j = 0; j < 100; j++) {
    A[i][j] = 0.0;
```

```
for (k = 0; k < 100; k++)
    A[i][j] += B[i][k] * D[k][j];
}
```

In C (row-major):

- **B[i][k]** accesses along a **row** → consecutive in memory (stride = 1).
- **D[k][j]** accesses along a **column** → not consecutive; elements separated by **row_size × element_size** bytes.
 - Here: row size = 100, double = 8 bytes → stride = $100 \times 8 = 800$ bytes.

Stride = distance in memory between consecutive elements of a vector.

In VMIPS:

- For non-unit stride (stride ≠ element size), you use **LVWS / SVWS**.

Problem:

- With non-unit stride, accesses may repeatedly hit the same memory bank ⇒ **bank conflicts**.

Rough idea for worst-case stride:

When **stride** is a multiple of **#banks**, the addresses always map to the same bank repeatedly → maximum conflicts.

Example from text:

- 8 memory banks, bank busy time = 6 cycles, total memory latency = 12 cycles.
- 64-element vector load, stride = 1:
 - Enough banks → $12 + 64 \approx 76$ cycles → ~ 1.2 cycles/element.
- Stride = 32 (multiple of 8 banks):
 - After the first access, every next access collides with a busy bank.
 - Each new access must wait 6 cycles.
 - Total $\approx 12 + 1 + 6 \times 63 = 391$ cycles → ~ 6.1 cycles/element.

So stride patterns can drastically change effective bandwidth.

12. Sparse matrices: gather and scatter

Sparse operations often look like:

```
c
for (i = 0; i < n; i++)
    A[K[i]] = A[K[i]] + C[M[i]];
```

K and **M** are index vectors pointing into **A** and **C**.

To vectorize this:

1. Load index vector **K** into **V_k**.
2. Use **LVI** (gather) to get **A[K[i]]** into a vector **V_a**.
3. Load index vector **M** into **V_m**.
4. Use **LVI** again to get **C[M[i]]** into **V_c**.
5. Do vector add: **V_a = V_a + V_c**.
6. Use **SVI** (scatter) to store **V_a** back to **A[K[i]]**.

This mechanism:

- **Gather**: convert sparse layout into dense vector in registers.
- Operate on it at high speed.
- **Scatter**: write updated dense vector back to sparse layout.

Caveat:

- Indexed loads/stores run slower than contiguous or stride-based loads/stores because:
 - Each element may go to a different, unpredictable bank.
 - You cannot group accesses nicely.
 - More conflicts, more latency.

GPUs essentially treat **all loads as gather**, **all stores as scatter**, and rely on programmers + hardware to **coalesce** accesses into unit-stride patterns to get good performance.

13. Programming model and vectorization levels

One big **advantage of vector architectures**:

- Very clear, static execution model.
- Compilers can tell you at **compile time**:
 - “This loop vectorizes.”
 - “This one doesn’t, because of X dependence/conditional/sparse access.”

Programmers and compilers can then **cooperate**:

- Programmer restructures code or adds hints (directives) where safe.
- Compiler uses this information to vectorize more loops correctly.

In practice, success depends strongly on:

- Algorithm choice,
- Data structure layout,
- How loops are written (nest ordering, dependencies).

The text mentions the **Perfect Club** benchmarks:

- Scientific programs running on a Cray Y-MP.
- Measured % of operations executed in vector mode:
 - Column 1: compiler optimization only.
 - Column 2: with expert programmer hints.

Key observations from the table:

- Without help: vectorization levels vary a lot (some as low as 4–15%, some above 90%).
- With hints: almost all codes get above 50% vectorization; many around 90%.
- Speedup from hints can be quite large (up to ~4×).

This shows:

- A *lot* of potential vector performance is unlocked when programmers collaborate with the compiler and adjust code to be “vector-friendly”.

14. Summary – tying it all together

Let's connect all the concepts:

1. SIMD vs MIMD

- SIMD handles *data-level parallelism* extremely well, with low instruction overhead and good energy efficiency.
- For future performance, SIMD is at least as important as adding more cores (MIMD).

2. Vector architectures are a powerful, general SIMD style:

- Work on entire vectors using vector registers.
- Hide latency with deep pipelines and streaming.
- Scale throughput with multiple lanes.
- Use:
 - **VLR** for arbitrary vector lengths and strip mining.
 - **VM** for handling conditionals in loops.
 - **Stride** and **gather/scatter** for multidimensional and sparse data.

3. Performance model:

- Break sequences into **convoys**.
- Time in **chimes**: $\#convoys \times vector_length$ (approx).
- Take into account start-up and memory system behavior.

4. Memory system:

- Multiple banks to sustain 1 word/cycle or more.
- Stride and bank mapping heavily impact performance.

5. Programming:

- Compilers can precisely tell what vectorizes.
- With modest programmer effort and hints, vectorization can be > 90% in many scientific codes, yielding big speedups.

Once you really understand this VMIPS model, it becomes much easier to:

- See why multimedia extensions (MMX/SSE/AVX) are like a **small, fixed-width** vector architecture.
- Understand why GPUs look like **massively parallel, masked/vector-like** processors with a different memory and control environment.

If you want, next we can:

- Take one specific subsection (e.g., vector masks or strip mining) and work through a numeric example with actual values, or
- Compare VMIPS vector instructions to real AVX/SVE instructions to see how this maps to modern CPUs.

I'll break this into clear sections so each idea from the text is unpacked and connected logically.

1. What are SIMD Instruction Set Extensions for Multimedia?

1.1 Motivation

Traditional general-purpose CPUs (e.g., 32-bit processors) were designed around:

- 32-bit integers
- 32-bit or 64-bit floating-point numbers

But **multimedia applications** (graphics, audio, video) often use *narrower* data types:

- **Graphics:**
 - Each pixel typically uses:
 - 8 bits for Red
 - 8 bits for Green
 - 8 bits for Blue
 - 8 bits for transparency/alpha
→ 32 bits per pixel, but each channel is only 8 bits.
- **Audio:**
 - Samples are often 8-bit or 16-bit.

Key observation:

If you have, say, a **256-bit register**, you can **pack many small values** into it:

- 32×8 -bit values
- 16×16 -bit values
- 8×32 -bit values
- 4×64 -bit values

If you partition the ALU and carry chains accordingly, a single instruction can operate on all these packed values in parallel.

This leads to **multimedia SIMD extensions**, where:

- One instruction = many element-wise operations on packed data within one register.

2. Basic Idea of Multimedia SIMD

2.1 Packed SIMD vs Vector Registers

Compare:

- **Vector architecture (like VMIPS):**
 - Few *big* vector registers, each holding many elements (e.g., 64×64 -bit).
 - A vector instruction operates on all active elements, vector length controlled by VLR.
- **Multimedia SIMD (MMX/SSE/AVX):**
 - Traditional scalar register file extended with **wider registers** (e.g., 64-bit, 128-bit, 256-bit).
 - Each register holds a **short fixed-length vector** of packed elements.
 - Example for 256-bit register:
 - 32×8 -bit
 - 16×16 -bit
 - 8×32 -bit
 - 4×64 -bit

Figure 4.8 (described):

- Shows typical SIMD operations for a 256-bit unit:
 - Unsigned add/sub: 8/16/32/64-bit packed elements
 - Max/min: same
 - Average, shifts, FP operations: same idea
- IEEE 754-2008 added support for:
 - 16-bit (half-precision)
 - 128-bit (quad-precision) floating-point

These SIMD instructions operate on **fixed-size vectors** determined by the register width and operand type.

3. Three Main Limitations vs True Vector Architectures

The text emphasizes that **multimedia SIMD extensions** (MMX, SSE, AVX) are **less elegant** and **less general** than classic vector architectures:

3.1 Fixed Vector Length (No Vector Length Register)

- **SIMD extensions:**
 - The number of data elements is **hard-coded** by:
 - The opcode (e.g., packed 8-bit, 16-bit, 32-bit)
 - The register size (64/128/256 bits)
 - Result:
 - To support more cases and operations, architectures like x86 have accumulated **hundreds of SIMD opcodes**.
- **Vector architectures:**
 - Have:
 - A **maximum vector length (MVL)** fixed by the hardware,
 - A **vector length register (VLR)** that specifies how many elements are active for this operation.
 - Benefits:
 - One instruction + VLR handles many lengths.
 - No need for many specialized opcodes.

Key consequence:

Vector ISAs are much cleaner and easier to retarget by compilers. SIMD multimedia ISAs are irregular and bloated with many variants.

3.2 Weaker Addressing Support (No Stride, No Gather/Scatter)

- **SIMD multimedia:**
 - Typically supports **only basic loads/stores**:
 - Load/store a fixed-width vector from a memory address (aligned/unaligned).
 - No built-in support for:
 - **Strided accesses** (e.g., taking every k-th element),
 - **Gather/scatter through index vectors** (sparse data, matrices).
- **Vector architectures:**
 - Provide:
 - **Strided load/store** (e.g. LVWS/SVWS in VMIPS),
 - **Indexed load/store (gather/scatter)** for sparse data.

Impact on compilers:

- Without strided and indexed memory operations:
 - Many loops that could be vectorized on a classic vector machine **cannot** be easily SIMD-ized.
 - Compiler is more limited; programmer has to reshape data to contiguous/packed form manually.

3.3 Lack of Architectural Mask Registers

- **Vector architectures:**
 - Have **vector mask registers** for per-element conditional execution.
 - This helps vectorize loops with `if` conditions inside.
- **Multimedia SIMD:**
 - Usually lacks **architectural** mask registers equivalent to VMIPS's VM.
 - Some SIMD sets do have comparison predicates and blending, but historically they were less uniform and less powerful than vector masks.

Result:

Conditional operations (e.g., masked computations) are harder and more awkward to express in multimedia SIMD than in vector architectures.

4. Evolution of x86 Multimedia SIMD Extensions

The section then traces how SIMD evolved on x86.

4.1 MMX (1996)

- First SIMD extension for x86.
- Reused the existing **64-bit floating-point registers** (x87 stack) as MMX registers:
 - 8×64 -bit registers.
- Supported:
 - 8×8 -bit or 4×16 -bit operations packed into each 64-bit register.
 - Parallel **MAX**, **MIN**, masking, DSP-style operations.
 - Ad hoc instructions designed for specific multimedia libraries.
- Used **existing floating-point load/store instructions** for memory access.

Downside: sharing registers with x87 meant:

- FP and MMX code could not be freely mixed without extra state management.
- It was a "first step" rather than a clean design.

4.2 SSE Family (1999–2007)

SSE = Streaming SIMD Extensions.

- **SSE (1999):**
 - Introduced separate **128-bit registers** (XMM0–XMM7 initially).
 - Could operate on:
 - 16×8 -bit
 - 8×16 -bit
 - 4×32 -bit
 - Added **parallel single-precision FP** operations.
 - Needed new data transfer instructions for XMM registers.
- **SSE2 (2001):**
 - Added **double-precision** FP SIMD.
 - Essentially allowed:
 - 2×64 -bit double-precision operations in parallel.
- **SSE3 (2004), SSE4 (2007):**
 - Incrementally extended the instruction set with more operations:
 - Better shuffle/permute, integer ops, special-purpose multimedia instructions.
 - Key idea: gradually increase computational density and support more codes.

With SSE and beyond:

- Code could achieve significantly higher peak FP performance if:
 - Data was arranged so that operands could be loaded into XMM registers as packed vectors.

4.3 AVX (2010)

AVX = Advanced Vector Extensions.

- Doubled the width of SIMD registers from 128 to **256 bits** (YMM registers).
- For 256-bit AVX:
 - **VADDPD**: add 4 packed double-precision elements.
 - **VMULPD**, **VDIVPD**, etc.
 - Vectors of:
 - 4×64 -bit doubles (DP)
 - 8×32 -bit floats (SP)
 - More narrow types as well.
- AVX also:
 - Introduced **VEX encoding** (more regular, 3-operand form),
 - Prepared the path for future 512-bit and even 1024-bit widths.

Figure 4.9 lists typical AVX instructions for double-precision:

- **VADDPD, VSUBPD, VMULPD, VDIVPD**
- **VFMADDPD, VFMSUBPD** (fused multiply-add/subtract)
- **VCMPxx** (packed comparisons)
- **VMOVBPD** (aligned moves)
- **VVBROADCASTSD** (broadcast a scalar double to all elements in a vector)

AVX also has **shuffle/permutation instructions**:

- To rearrange elements within 256-bit registers:
 - Permute 32-bit, 64-bit, or 128-bit fields.
- Helps compensate for the lack of true vector addressing modes by allowing **register-level data rearrangement**.

5. Why Are These Multimedia SIMD Extensions Popular Despite Their Limitations?

Even though they're less elegant than vector architectures, they have **several practical advantages**:

1. Low implementation cost

- Partitioning adders and multipliers inside existing ALUs is relatively cheap.
- You reuse the same hardware units with extra wiring and control.

2. Small extra architectural state

- Each SIMD extension adds only a handful of wide registers.
- This keeps **context switch time** reasonable.

3. Less demanding memory bandwidth

- True vector machines assume very high memory bandwidth (many elements per cycle).
- General-purpose systems often can't justify that cost.
- SIMD operates on short fixed-width vectors and usually does one load/store per instruction, easing bandwidth pressure.

4. Simpler interaction with virtual memory

- A vector instruction might, in principle, access dozens of elements, crossing page boundaries.
- If a page fault happens in the middle, handling it is complex.
- Multimedia SIMD:
 - Typically uses one load per vector operand.
 - Operands are aligned and don't cross page boundaries.
 - No single instruction generates a large unknown number of page faults.

5. Flexibility for new media standards

- Short fixed-size vectors make it easier to add:
 - Special permutation operations,
 - Instructions that take unusual operand counts,
 - Operations tailored to specific codecs or formats.

6. Historical skepticism about vector engines with caches

- Earlier vector machines sometimes interacted poorly with cache hierarchies.
- This made architects cautious.
- More recent vector architectures have fixed many issues, but the **legacy perception** helped push SIMD-style solutions instead.

In practice, these extensions are *incremental*, *cheap*, and *backwards-compatible*, which is very attractive in the x86 ecosystem.

6. Example: DAXPY with 256-bit MIPS SIMD vs Scalar

The book gives an example to illustrate how a **SIMD DAXPY** loop looks on a hypothetical MIPS with **256-bit SIMD** (4 doubles per instruction).

6.1 Scalar MIPS DAXPY (concept recap)

We already saw earlier a scalar loop like:

```
asm

L.D F0, a          ; load scalar a
DADDIU R4, Rx, #512 ; end address (for 64 elements * 8 bytes)
Loop:
  L.D  F2, 0(Rx) ; load X[i]
  MUL.D F2, F2, F0
  L.D  F4, 0(Ry) ; load Y[i]
  ADD.D F4, F4, F2
  S.D  F4, 0(Ry) ; store Y[i]
  DADDIU Rx, Rx, #8
  DADDIU Ry, Ry, #8
  DSUBU R20, R4, Rx
  BNEZ  R20, Loop
```

This processes **one element per iteration**.

6.2 MIPS SIMD with 256-bit Registers (4 doubles per instruction)

Assumptions:

- We have 256-bit SIMD instructions that operate on **4 doubles at once**:
 - **L.4D, MUL.4D, ADD.4D, S.4D**
 - Each operates on 4 DP elements in parallel.

Key changes in code:

```
asm
```

```
L.D F0, a          ; load scalar a
MOV F1, F0          ; copies of 'a' for other lanes
MOV F2, F0
MOV F3, F0

DADDIU R4, Rx, #512 ; last address (64 elements * 8 bytes = 512 bytes)

Loop:
  L.4D F4, 0(Rx)    ; load X[i..i+3]: F4,F5,F6,F7
  MUL.4D F4, F4, F0  ; multiply each lane by a
  L.4D F8, 0(Ry)    ; load Y[i..i+3]: F8,F9,F10,F11
  ADD.4D F8, F8, F4  ; add each lane
  S.4D F8, 0(Rx)    ; store Y[i..i+3]

  DADDIU Rx, Rx, #32 ; 4 doubles * 8 bytes = 32
  DADDIU Ry, Ry, #32
  DSUBU  R20, R4, Rx
  BNEZ   R20, Loop
```

Notes:

- The increment changed from 8 to 32 because each iteration handles 4 doubles (4×8 bytes).
- Some extra **MOV** instructions are used to replicate scalar **a** across multiple registers so each lane has the appropriate value (in practice, real SIMD ISAs have a **broadcast** instruction).
- Each SIMD instruction processes 4 elements at once → fewer loop iterations.

6.3 Instruction Count Comparison

- Scalar MIPS: about **578** dynamic instructions.
- SIMD MIPS: about **149** dynamic instructions.

That's a $\approx 4\times$ reduction (not as dramatic as full VMIPS vectorization's $100\times$, but still significant).

7. Programming Multimedia SIMD Today

Historically:

- SIMD multimedia extensions were **ad hoc**, irregular, and full of special-case instructions.
- Therefore, they were primarily used:
 - Through **hand-tuned libraries** (e.g., BLAS, multimedia libraries),
 - Or **hand-written assembly**.

Recently:

- Extensions became more regular (esp. AVX, AVX2, AVX-512).
- Compilers have incorporated **auto-vectorization** techniques from classic vectorizing compilers.
- Many modern compilers (e.g., for C/C++/Fortran) can:
 - Automatically generate SSE/AVX code for loops with clear data-level parallelism.

However, programmers must:

- Ensure **data alignment** to match SIMD width (e.g., 16-byte or 32-byte align).
- Respect aliasing and alignment rules to avoid conservative fallback to scalar.

Otherwise:

- The compiler may refuse to generate SIMD instructions and revert to scalar code, losing performance.

8. The Roofline Performance Model

The Roofline model is a visual tool for understanding the performance potential of a machine for a given kernel, considering both:

- **Compute throughput** (FLOP/s),
- **Memory bandwidth** (Bytes/s),
- And **arithmetic intensity**.

8.1 Arithmetic Intensity (AI)

Arithmetic intensity =

$$AI = \frac{\text{Total floating-point operations}}{\text{Total bytes transferred to/from main memory}}$$

- Units: FLOPs per byte.
- It depends on:
 - Algorithm,
 - Implementation,
 - Data structure,
 - But **not** on hardware.

Examples (Figure 4.10 style):

- Low AI:
 - Sparse matrix–vector multiply (SpMV),
 - Simple stencil codes (structured grids).
- Medium AI:
 - Spectral methods (FFTs).
- High AI:
 - Dense matrix–matrix multiply (BLAS3),
 - N-body simulations (particle methods).

Some kernels have AI that **scales with problem size** (e.g., dense matrix multiply), others have **fixed AI** independent of problem size (e.g., SpMV).

8.2 Roofline Plot Axes

On the Roofline plot (Figure 4.11):

- **X-axis:** Arithmetic Intensity (FLOP/byte), typically log scale.
- **Y-axis:** Achievable floating-point performance (GFLOP/s), also log scale.

You draw two key lines:

1. **Horizontal line:** peak floating-point performance (hardware limit).

- E.g., for NEC SX-9: 102.4 GFLOP/s.
- For Intel Core i7 920: 42.66 GFLOP/s.

2. **Diagonal 45° line:** memory bandwidth limit.

- Slope given by **peak memory bandwidth**:
 - NEC SX-9: 162 GB/s (from Stream benchmark).
 - Core i7 920: 16.4 GB/s.

Why 45°?

Because:

$$\text{Attainable GFLOP/s} = \text{Memory Bandwidth (GB/s)} \times \text{Arithmetic Intensity (FLOP/byte)}$$

So at AI = 1, achievable GFLOP/s = bandwidth (in GB/s). This is a straight line in log-log.

8.3 The Roofline Itself

The achievable performance is:

$$\text{Performance} \leq \min(\text{Peak Floating-Point Perf}, \text{Memory Bandwidth} \times \text{AI})$$

Graphically:

- The **roof** is formed by:
 - The diagonal line (memory-bound region),
 - The horizontal line (compute-bound region),
 - They meet at a **ridge point**.

Interpretation:

- If your kernel's AI (x-position) lies:
 - **Left of the ridge** → memory-bound.
 - **Right of the ridge** → compute-bound.

Drawing a **vertical line** at your kernel's AI:

- The maximum possible performance lies at the intersection with the roof.

8.4 Ridge Point and Its Significance

The **ridge point** is where:

$$\text{Peak FLOP/s} = \text{Bandwidth} \times \text{AI}_{\text{ridge}}$$

So:

$$\text{AI}_{\text{ridge}} = \frac{\text{Peak FLOP/s}}{\text{Bandwidth (Bytes/s)}}$$

If:

- Ridge point far to the **right**:
 - Only very high-intensity kernels can reach peak computational throughput.
 - Most real workloads will be memory-bound.
- Ridge point far to the **left**:
 - Many kernels, even with modest AI, can reach peak throughput.
 - Indicates a **well-balanced** machine with high memory bandwidth.

8.5 NEC SX-9 vs Intel Core i7 920 (Example from Figure 4.11)

From the text:

- **NEC SX-9 (vector supercomputer):**
 - Peak DP performance: 102.4 GFLOP/s.
 - Peak memory bandwidth (Stream): 162 GB/s.
- **Intel Core i7 920 (Nehalem, multicore with SIMD):**
 - Peak DP performance: 42.66 GFLOP/s.
 - Peak memory bandwidth (Stream): 16.4 GB/s.

Comparison:

- Compute:
 - SX-9 is 2.4× faster (102.4 vs 42.66 GFLOP/s).
- Memory:
 - SX-9 is ~10× faster in bandwidth (162 vs 16.4 GB/s).

For a kernel with **AI = 0.25 FLOP/byte**:

- On Core i7:
 - Max GFLOP/s = 16.4 GB/s × 0.25 FLOP/byte ≈ 4.1 GFLOP/s.
- On SX-9:
 - Max GFLOP/s = 162 GB/s × 0.25 ≈ 40.5 GFLOP/s.

So SX-9 is **10× faster** for low-AI workloads (memory-bound region).

Ridge points:

- Core i7 ridge AI ≈ 2.6 FLOP/byte.
- SX-9 ridge AI ≈ 0.6 FLOP/byte.

Interpretation:

- On Core i7, only kernels with AI $> \sim 2.6$ can reach 42.66 GFLOP/s.
- On SX-9, even kernels with AI ≈ 0.6 can potentially reach 102.4 GFLOP/s.
- This shows how **vector machines with high bandwidth** are much better for memory-intensive workloads.

9. Overall Takeaways from Section 4.3

1. **SIMD multimedia extensions** (MMX, SSE, AVX) brought SIMD into mainstream CPUs:
 - Operate on packed narrow data types inside wide registers.
 - Improve media and floating-point performance with relatively low implementation cost.
2. They are **less general** than classic vector architectures because:
 - Fixed-length vectors (no VLR),
 - Weak addressing modes (no stride/gather/scatter),
 - Historically weaker masking mechanisms.
3. They are still popular because:
 - Cheap to add,
 - Minimal extra architectural state,
 - Easier to integrate with caches and virtual memory,
 - Flexible for evolving media standards.
4. Compilers are increasingly able to **auto-generate** SIMD code, especially for FP-heavy workloads, provided:
 - Data is well aligned and vectorizable.
5. The **Roofline model** provides a clean, visual way to:
 - Relate SIMD / vector compute power,
 - Memory bandwidth,
 - Arithmetic intensity,
 - And determine whether a kernel is compute-bound or memory-bound.
6. Vector supercomputers like the NEC SX-9 illustrate:
 - How very high bandwidth plus vectorization can massively outperform general-purpose SIMD machines like Core i7, especially for low-AI workloads.

If you'd like, next we can:

- Take a concrete kernel (e.g., SpMV or dense GEMM), compute its arithmetic intensity, and place it on the Roofline graph, or
- Contrast programming with AVX intrinsics vs a true vector ISA like RISC-V V for the same operation.

I'll walk through this section the same way: first the **CUDA programming model**, then the **GPU hardware model**, then **PTX**, and finally the **DAXPY PTX** and **memory coalescing** idea.

1. Why GPUs Matter for Computing

- For a few hundred dollars, you can buy a GPU with **hundreds or thousands of floating-point units**.
- That makes **massively parallel computation** accessible to ordinary developers.
- Once a proper programming model arrived (CUDA, then OpenCL), people started using GPUs not only for graphics but also for:
 - Scientific computing,
 - Machine learning,
 - Multimedia applications, etc.

Key point:

GPUs are not "CPU variants". Their lineage is from **graphics accelerators**, not from general-purpose CPUs. They must **still** be excellent at graphics, so their compute design is heavily constrained by:

- How graphics pipelines work,
- Massive parallelism,
- High throughput over low latency.

The question GPU architects ask is essentially:

"We already have a ton of hardware to draw graphics. How can we expose that same hardware to speed up general-purpose computations?"

2. CUDA Programming Model – Conceptual View

NVIDIA's CUDA is a **C/C++-like language + runtime** for programming GPUs. The design goals:

- Make heterogeneous programming (CPU + GPU) more manageable.
- Provide a clean abstraction over multiple levels of parallelism:
 - Instruction-level,

- SIMD-style (SIMT),
- Multithreading,
- Multi-core (multiple GPU “multiprocessors”).

2.1 Host vs Device

In CUDA terminology:

- **Host** = CPU + system memory.
- **Device** = GPU + GPU memory.

You write:

- Normal C/C++ code for the host, marked with `__host__`.
- GPU kernel code, marked with `__device__` or `__global__`.

Roughly:

- `__global__` = function callable from host, executes on device (kernel).
- `__device__` = function callable from device, executes on device.

Variables inside `__device__` / `__global__` code are allocated in **GPU memory** (or GPU-resident spaces like shared memory, registers, etc.).

2.2 Kernel Launch Syntax

CUDA extends function calls with a special syntax:

```
cpp
kernel<<<dimGrid, dimBlock>>>(...);
```

- **dimGrid**: number of **thread blocks** in the grid (1D/2D/3D).
- **dimBlock**: number of **threads per block** (1D/2D/3D).

Inside the kernel, CUDA provides built-in variables:

- **blockIdx** – index of the current **block**.
- **threadIdx** – index of the current **thread within its block**.
- **blockDim** – dimension (size) of the block.

Together, these let each thread compute a **global index** into arrays.

2.3 Example: DAXPY in C vs CUDA

The original scalar C DAXPY:

```
c

// Invoke DAXPY
daxpy(n, 2.0, x, y);

// DAXPY in C
void daxpy(int n, double a, double *x, double *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a * x[i] + y[i];
}
```

This is a **simple for-loop**; each iteration is independent.

CUDA version:

```
cpp

// Host code: launch DAXPY with 256 threads per block
__host__
int nblocks = (n + 255) / 256;
daxpy<<<nblocks, 256>>>(n, 2.0, x, y);

// Device code: DAXPY kernel
__device__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        y[i] = a * x[i] + y[i];
}
```

Step-by-step:

1. The host code computes number of blocks:

$$\text{nblocks} = \left\lceil \frac{n}{256} \right\rceil = \frac{n + 255}{256}$$

2. It launches a kernel with:

- **nblocks** blocks,
- **256** threads per block.

3. Each CUDA thread computes its own index:

```
cpp

i = blockIdx.x * blockDim.x + threadIdx.x;
```

So, thread (blockIdx.x, threadIdx.x) is responsible for element i.

4. **if (i < n)** guards against extra threads beyond the data length.

Pattern:

- **Each loop iteration** in the original C code becomes **one CUDA thread**.
- The programmer explicitly exposes parallelism:
 - No loop in device code (except the **if (i < n)** guard).

- Each thread completes its own iteration.

2.4 Thread Blocks and Scheduling Constraints

CUDA imposes some constraints to simplify scheduling:

- **Thread Blocks** must be **independent**:
 - They can execute in any order.
 - They can be paused and resumed independently.
 - They should not depend on each other's completion (except via global memory with atomics).
- Threads *within* a block can:
 - Communicate via **shared memory**,
 - Synchronize via `__syncthreads()` (barrier).

Technically:

- GPU hardware handles scheduling, execution, and management of threads and blocks.
- The OS is more or less uninvolved in fine-grained scheduling on the device.

For performance:

- Programmers must:
 - Keep groups of **32 threads** (warps) in the same control path to avoid divergence.
 - Launch **many more threads** than the number of cores to hide DRAM latency.
 - Make memory accesses **coalesced** (adjacent threads accessing adjacent addresses).

CUDA often exposes **intrinsics** and options that let you tune things like:

- Cache usage,
- Synchronization,
- Atomic operations,
- Warp shuffles, etc.

This is part of the classic **productivity vs performance** trade-off: high-level, easy model vs low-level, fully exposed hardware control.

3. GPU Hardware Model (NVIDIA-style, Fermi-generation example)

Now we map the **software abstractions** (Grid, Block, Thread) to **hardware structures**.

3.1 Terminology Mapping (Figure 4.12)

The textbook introduces a “clean” naming scheme and then maps it to NVIDIA's jargon. Rough mapping:

Program abstractions:

- **Grid** (CUDA term):
 - Our descriptive term: *Vectorizable Loop*.
 - Meaning: the entire GPU kernel launch → conceptually one big vectorizable loop over data.
- **Thread Block:**
 - Our descriptive term: *Body of a (strip-mined) vectorized loop*.
 - Contains many threads, can use shared memory.
- **CUDA Thread:**
 - Our descriptive term: *one iteration of the scalar loop*.
 - Corresponds to operations on a single element.

Machine objects and hardware:

- **Warp:**
 - Our descriptive name: *Thread of SIMD instructions*.
 - A group of 32 CUDA threads executed **in lockstep** on SIMD hardware.
- **PTX Instruction:**
 - Our descriptive name: *Vector/SIMD instruction*.
 - Operates on all lanes of a warp at once.
- **Multithreaded SIMD Processor:**
 - CUDA/NVIDIA term: **Streaming Multiprocessor (SM)**.
 - Think of this like a “small vector processor”:
 - Has multiple SIMD lanes,
 - A warp scheduler,
 - Register file,
 - Shared memory.
- **Thread Block Scheduler:**
 - NVIDIA term: **GigaThread Engine**.
 - Distributes thread blocks to SMs.
- **Warp Scheduler:**
 - Schedules which warp runs which instruction next on an SM.
- **SIMD Lane:**
 - NVIDIA term: **Thread Processor**.
 - Executes one element (one thread’s work) of a warp for each SIMD instruction.

Memory:

- **Global Memory:**

- GPU DRAM accessible by all SMs.
- **Local Memory:**
 - Private per-thread memory (like a stack or thread-local).
- **Shared Memory:**
 - On-chip SRAM, shared by threads in one block (within a single SM).
- **Registers:**
 - Per-SM register file, divided across lanes and allocated to threads.

3.2 Example: Vector Multiply of Length 8192

Consider computing:

$$A[i] = B[i] \cdot C[i], \quad i = 0 \dots 8191$$

Each vector length = 8192.

Mapping to GPU:

- A **Grid** corresponds to the whole 8192-element operation.
- Break into **Thread Blocks:**
 - Suppose each block handles **512 elements**.
 - Then we need $8192 / 512 = 16$ thread blocks.
- Within each Thread Block:
 - Threads are grouped into warps of 32 threads.
 - Each warp operates on **32 elements at a time**.

So:

- Grid = 16 thread blocks.
- Each block = 512 CUDA threads = 512 data elements.
- Each warp = 32 CUDA threads = 32 elements.
- Number of warps per block = $512 / 32 = 16$ warps.

Each SM gets assigned one or more thread blocks by the **Thread Block Scheduler**.

3.3 Multithreaded SIMD Processor (Streaming Multiprocessor)

Figure 4.14 shows a simplified diagram of an SM (“multithreaded SIMD Processor”):

- Components:
 - **Warp scheduler with a scoreboard:**
 - Tracks many warps (e.g., up to 48),
 - Decides which warp’s next instruction to issue each cycle.

- **Instruction cache and instruction register.**
- **Lots of registers:**
 - Example: 32,768 32-bit registers per SM.
- **SIMD lanes:**
 - For Fermi, 16 physical lanes per SM.
- **Load/store units:**
 - Several units that access memory.
- **Local (shared) memory:**
 - E.g., 64 KB on-chip memory per SM, shared by threads in that block.
- **Address coalescing unit:**
 - Combines multiple per-lane loads/stores into fewer memory transactions when addresses are contiguous.

Each SM executes **many warps** interleaved in time to:

- Hide memory latency,
- Keep its functional units busy.

3.4 Two-Level Hardware Scheduling

There are essentially **two scheduling layers** in the GPU:

1. Global: Thread Block Scheduler (GigaThread Engine)

- Receives all the thread blocks of a grid from the host.
- Chooses which SM gets which block.
- Ensures blocks are placed where:
 - Enough registers are available,
 - Enough shared memory is available,
 - Data can be efficiently accessed.

2. Local: Warp Scheduler (per SM)

- Maintains a pool of active warps (e.g., up to 48).
- Uses a **scoreboard** to track:
 - Which warp is stalled (e.g., waiting on memory),
 - Which warp is ready.
- Issues one instruction from some ready warp per cycle (or more, depending on architecture).
- Can interleave warps arbitrarily to hide latency.

Because each warp and each thread block is **independent**, the GPU can reschedule freely.

3.5 SIMD Width and Lanes on Fermi

- Warp size = 32 threads (SIMD width = 32).
- Physical hardware lanes per SM on Fermi: **16 lanes**.
 - So a 32-wide warp is executed over **two cycles**:
 - First cycle: first 16 threads,
 - Second cycle: remaining 16 threads.

Analogy to vector processor:

- Lanes = 16.
- Vector length (= warp size) = 32.
- One “chime” (full warp execution) = 2 cycles.

The SIMT execution model:

- All 32 threads in a warp run the **same instruction** at the same time (on different data).
- If threads diverge due to control flow (**if, else**), the warp executes both paths with some threads masked off (we’ll come back to that later in the chapter).

3.6 Register File and Per-Thread View

- Per SM: 32,768 32-bit registers (Fermi example).
- Each thread can use up to **64 registers** (compiler-limited).

You can think of it as:

- Each CUDA thread has its own set of scalar registers, but:
- From the vector perspective:
 - Each **warp** has a set of **vector registers**:
 - Each vector register has 32 elements (one per thread).
 - Each element is 32 bits (or 64 bits by pairing two 32-bit registers).

For double precision:

- Each double uses two 32-bit registers.
- So a thread using doubles effectively has fewer DP registers.

The hardware allocates physical registers to warps at runtime; when a warp finishes, those registers are freed.

Important:

- A **CUDA Thread** is *not* like a heavy POSIX thread:
 - Very lightweight,

- No OS scheduling,
- No arbitrary system calls.

4. PTX – NVIDIA's Virtual Instruction Set

PTX = **Parallel Thread Execution**.

Design goals:

- Provide a stable, **virtual ISA** for compilers.
- Hide hardware-specific instruction set details.
- Allow the **same PTX** to be re-optimized for different GPU generations.

Key points:

- Compilers emit PTX.
- A later tool (ptxas / driver JIT) compiles PTX to **hardware instructions**.
- PTX uses **virtual registers** (R0, R1, RD0, etc.), not physical registers.
- One PTX instruction:
 - Often maps 1:1 to hardware instructions,
 - But can sometimes expand to multiple hardware instructions or be optimized away.

Format:

```
text

opcode.type d, a, b, c;
```

- **opcode** = operation (add, mul, ld, st, etc.).
- **.type** = data type:
 - **.b8, .b16, .b32, .b64** = untyped bits.
 - **.u8, .u16, .u32, .u64** = unsigned int.
 - **.s8, .s16, .s32, .s64** = signed int.
 - **.f16, .f32, .f64** = floating point.
- **d** = destination register.
- **a, b, c** = source operands (registers or constants).

Instructions can be **predicated**:

- Prepend with **@p** where **p** is a predicate register (1-bit).
- Predicates set via **setp** instructions.

4.1 Instruction Groups (Figure 4.17)

Categories include:

- **Arithmetic:**
 - `add`, `sub`, `mul`, `mad` (multiply-add), `div`, `rem`, `abs`, `neg`, `min`, `max`, `setp`, `mov`, `sefp`, `cvt`.
- **Special Function:**
 - `rcp`, `sqrt`, `rsqrt`, `sin`, `cos`, `lg2`, `ex2`.
- **Logical / Bitwise:**
 - `and`, `or`, `xor`, `not`, `cnot`, `shl`, `shr`.
- **Memory Access:**
 - `ld` (load), `st` (store) with spaces: `.global`, `.shared`, `.local`, `.const`.
 - `tex` for texture lookups.
 - `atom` for atomic operations.
- **Control Flow:**
 - `bra` (branch),
 - `call`, `ret`,
 - `bar.sync` (barrier within block),
 - `exit` (end of thread).

PTX describes **what one CUDA thread does**. The GPU hardware then executes this PTX stream **SIMT-style** across all threads in a warp.

5. DAXPY in PTX (One Thread's View)

The PTX sequence for **one thread's iteration** of DAXPY roughly looks like:

```
asm

shl.u32    R8, blockIdx, 9      ; R8 = blockIdx * 2^9 = blockIdx * 512
add.u32    R8, R8, threadIdx    ; R8 = i = global thread index
shl.u32    R8, R8, 3           ; byte offset = i * 8 (double is 8 bytes)

ld.global.f64 RD0, [X+R8]      ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8]      ; RD2 = Y[i]
mul.f64    RD0, RD0, RD4        ; RD0 = RD0 * RD4 (RD4 holds scalar a)
add.f64    RD0, RD0, RD2        ; RD0 = RD0 + RD2 (X[i]*a + Y[i])
st.global.f64 [Y+R8], RD0      ; Y[i] = result
```

Explanation:

1. `shl.u32 R8, blockIdx, 9`:

- $2^9 = 512$, so multiplying block index by 512.
- This is the number of threads per block → gives the starting index for this block.

2. `add.u32 R8, R8, threadIdx`:

- Add the thread's local index to get the global index `i`.

3. `shl.u32 R8, R8, 3`:

- Each double is 8 bytes = 2^3 , so shifting left by 3 multiplies by 8.
- Now `R8` holds the **byte offset** for element `i`.

4. `ld.global.f64`:

- Load `X[i]` and `Y[i]` into 64-bit registers `RD0` and `RD2`.

5. `mul.f64` and `add.f64`:

- Perform floating-point multiply and add.

6. `st.global.f64`:

- Store the result back to `Y[i]`.

The `if (i < n)` from the CUDA source would be implemented with a predicate:

- Compute `i`.
- Compare `i < n` → set predicate.
- Guard the load/mul/add/store with that predicate or branch over them.

6. Memory Access: Gather/Scatter and Coalescing

Important conceptual difference vs vector architectures:

- In **classic vector processors**, you have separate instructions for:
 - Unit-stride vector loads/stores,
 - Strided loads/stores,
 - Indexed gather/scatter.
- In **GPUs**, all memory accesses are conceptually **gather/scatter-like**:
 - Each lane (thread in a warp) gives an independent address.
 - Hardware must decide how to group these into memory transactions.

To achieve high bandwidth, GPUs use:

6.1 Address Coalescing

- The **Address Coalescing Unit** looks at the addresses issued by all lanes in a warp for a load/store.
- If it detects that:
 - Threads in a warp are accessing **consecutive or nearby addresses**, and
 - Those addresses fall into a small number of memory blocks,
- It merges these accesses into a small number of **block transactions** (e.g., one 32-word transaction).

For example:

- In our DAXPY:
 - Threads 0..31 have $i = 0..31$.
 - Offsets: $0 \times 8, 1 \times 8, 2 \times 8, \dots, 31 \times 8 \rightarrow$ strictly increasing by 8 bytes.
 - These are perfectly coalesced: one or a few burst transfers from global memory.

If threads access memory in a scattered or irregular pattern:

- Coalescing fails,
- Each lane might require its own memory transaction,
- Bandwidth collapses, latency increases.

So GPU programming best practice:

- **Assign consecutive data elements to consecutive threads** (as in our DAXPY example).
- Ensure **aligned** data structures.
- Strive for **unit-stride** or near-unit-stride accesses across warps.

That's how GPUs recover **vector-like unit-stride bandwidth** despite having only one "kind" of load/store in the ISA.

7. Summary of Section 4.4

Putting it all together:

1. **GPUs** evolved from graphics accelerators and offer massive FP throughput at low cost.
2. **CUDA** provides a C-like programming model where:
 - The host launches a kernel over a **Grid of Thread Blocks**.
 - Each block has many **threads** that execute the same code but on different data elements.
 - Each loop iteration in a data-parallel loop usually maps to one CUDA thread.
3. **Hardware model** (Fermi example):
 - Many **Streaming Multiprocessors** (multithreaded SIMD Processors).
 - Each SM has:
 - A **warp scheduler**, **SIMD lanes**, large register file, **shared memory**.
 - A **Thread Block Scheduler** assigns thread blocks to SMs.
 - Within each SM, **warps** of 32 threads run SIMT-style across the lanes.
4. **PTX** is a virtual ISA:
 - Stable target for compilers,
 - Uses virtual registers and typed instructions,
 - Later lowered to actual GPU machine code.
5. **DAXPY** in PTX:

- Each thread computes its own index based on `blockIdx`, `blockDim`, `threadIdx`.
- Then loads, computes, and stores one element's result.

6. Memory model:

- All loads/stores are conceptually gather/scatter.
- Performance depends heavily on **address coalescing**:
 - Adjacent threads accessing adjacent memory locations,
 - Enabling large contiguous memory transactions.

If you want, next we can dive into:

- How GPUs handle **branch divergence** (masks and SIMT), or
- Compare a CUDA kernel + PTX for something more complex, like matrix multiplication.

Below is a **fully detailed, structured, and complete explanation** of the section “*Conditional Branching in GPUs*” (and the memory/architecture continuation up to the summary) exactly in the same style I used earlier—clear, deep, and without skipping any technical point.

Conditional Branching in GPUs — Full Detailed Explanation

GPUs support conditional branching (IF/ELSE statements, loops, switch cases, function calls), but they handle them **very differently** from CPUs or even traditional vector processors. This is because a GPU executes **SIMD-style groups of 32 threads (warps)**, where the same instruction is broadcast to all lanes at once.

When threads in a warp take **different control-flow paths**, this is called **divergence**, and GPU hardware must manage it. To do this efficiently, GPUs use:

- Predicate registers,
- Per-lane masks,
- A branch synchronization stack,
- Special instructions the compiler inserts,
- Instruction markers that help detect divergence and reconvergence.

We will walk through all of these mechanisms in detail.

1. Branching at the PTX Level vs. Hardware Level

1.1 PTX (assembler) view

In PTX, you write:

- `bra target` (branch)

- `call, ret, exit`
- Predicate-based execution, e.g. `@P1 add.f32 ...`

PTX appears to allow each **thread** to branch independently, as if each had its own program counter.

But this is an illusion.

Hardware executes in **warp-wide lockstep**, meaning:

- All 32 threads in a warp execute the same instruction,
- Or they are **masked off** (disabled) for that instruction.

This is why conditional execution on a GPU is not trivial.

1.2 Hardware view

At the GPU machine-instruction level, the hardware supports:

- Branches and jumps,
- Calls and returns,
- Indexed branches and calls,
- Exit instructions,
- Special **branch synchronization instructions**,
- A **per-warp branch synchronization stack**,
- Per-lane predicate bits (enable/disable).

Each active warp maintains:

- **Active mask** → which lanes participate in the current instruction.
- **Branch stack entries**:
 - *A token (ID),*
 - *A target instruction address,*
 - *A target active mask.*

Branch instructions push or pop entries on this synchronization stack.

This system allows divergent branch paths to be:

- Executed **serially**, with some lanes masked off,
- Then **reconverged** at the correct program point.

2. How GPUs Execute IF / THEN / ELSE

GPUs can handle an IF statement in two main ways:

2.1 Case 1 — Compiler converts to predicated instructions (no branch)

For **simple IF/ELSE** constructs, the PTX assembler often translates the entire conditional block into **predicated instructions**, eliminating branches entirely.

Example:

- Compute predicate **P1** = (X[i] != 0)
- THEN block executes as @P1
- ELSE block executes as @!P1

So each instruction is broadcast to the whole warp, but:

- Lanes where predicate = 1 execute,
- Lanes where predicate = 0 do nothing.

This avoids divergence entirely.

Efficiency consequence

If THEN and ELSE paths are equal-length:

- Half the lanes are active in THEN,
- The other half are active in ELSE,
- So performance ≈ **50% efficiency**.

2.2 Case 2 — Complex IF statements → actual branching + stack

When PTX detects:

- Nested IFs,
- Loops with exits,
- Switch statements,
- Function calls,

it emits real machine-level branching code using:

- Special stack operations,
- Mask modification instructions,
- Reconvergence markers.

3. Understanding Divergence and Reconvergence

When does divergence occur?

A branch **diverges** when threads in a warp do **not** agree on the branch condition.

Example:


```
c
if (X[i] != 0)
    ...
else
    ...
```

If some threads have $X[i] \neq 0$ and others have $X[i] = 0$:

- Some lanes take the THEN path,
- Others take the ELSE path,
- Warp cannot proceed in lockstep.

How the hardware handles divergence

When encountering such a branch:

1. The hardware pushes the **current active mask** onto the branch synchronization stack.
2. The active mask is updated to include only lanes that take the branch.
3. The branch target executes with only that subset active.
4. After completion, a special marker:
 - Pops the stack,
 - Restores the previous active mask (but often complemented),
 - Executes the ELSE part with the opposite mask.
5. At the end, another marker pops the previous mask to restore full warp activity.

Efficiency drops with nested branches

If you have:

- One nested IF $\rightarrow$ ~50% efficiency
- Two nested IFs $\rightarrow$ ~25%
- Three nested IFs $\rightarrow$ ~12.5%

This happens because fewer lanes remain active as nesting increases.

This is identical to vector processors where mask registers allow sparse lane activation.

4. Example: PTX Translation of IF/ELSE

Given:

```
c
if (X[i] != 0)
    X[i] = X[i] - Y[i];
else
    X[i] = Z[i];
```

The PTX assembler may generate something like:

```
asm

ld.global.f64 RD0, [X+R8]          ; load X[i]
setp.neq.s32 P1, RD0, #0          ; predicate P1 = (X[i] != 0)

@!P1 bra ELSE1, *Push             ; divergent branch → push old mask, set new mask

ld.global.f64 RD2, [Y+R8]
sub.f64 RD0, RD0, RD2
st.global.f64 [X+R8], RD0

@P1 bra ENDIF1, *Comp             ; complement mask for ELSE path

ELSE1:
ld.global.f64 RD0, [Z+R8]
st.global.f64 [X+R8], RD0

ENDIF1:
<next instruction>, *Pop          ; restore previous mask
```

Where:

- ***Push** saves mask for the divergent branch,
- ***Comp** complements mask for ELSE block,
- ***Pop** restores the old mask at reconvergence.

This mechanism allows nested branches, loops, and complex control flow while maintaining warp semantics.

5. Performance Implications of Divergence

The key restriction:

A warp can execute only **one instruction** at a time.
Lanes either perform that instruction or remain idle.

So if divergence splits the warp:

- Both paths must be executed **serially**,
- Mask selects which lanes are active during each part.

Worst case:

- Only 1 out of 32 lanes active → ~3% efficiency
- The other 31 lanes idle.

This is why:

- SIMT appears to give per-thread flexibility,
- But true performance lies in keeping warps converged (all lanes executing same path).

6. GPUs Handle Loop Boundaries Using the Same Mechanism

For grid-stride loops, strip-mining, loops that end unevenly:

- GPU uses internal masks to disable lanes whose loop iteration count is exceeded.
- On last iteration, some lanes become inactive.
- Reconvergence naturally occurs when all are done.

This eliminates the need for explicit vector length registers (like in traditional vector architectures).

7. Memory Structure in GPUs

Three levels of memory exist in a GPU (Figure 4.18):

7.1 Private Memory (per-thread)

- Located in DRAM.
- Used for:
 - Register spills,
 - Thread-local variables,
 - Stack frames.
- Cached in L1 and L2.

Each lane has private memory distinct from all others.

7.2 Local Memory (per-block shared memory)

- On-chip SRAM (fast, low-latency).
 - Shared among all threads in one block (but not across blocks).
 - Used for:
 - Shared variables,
 - Tile-based algorithms,
 - Inter-thread communication.
-

7.3 Global GPU Memory

- Large off-chip DRAM,
- Visible to all blocks and grids,
- Accessible by host CPU,

- High latency but high bandwidth.

Only this memory persists across kernel launches unless explicitly managed.

8. Memory Latency Philosophy: Vector vs GPU

Vector architectures

- Hide latency through pipelined multi-element loads.
- One vector load brings in many lanes worth of data.

GPUs

- Hide latency via massive **multithreading**.
 - Thousands of warps available; when one stalls, another runs.
 - Cache hierarchy added later (Fermi and beyond) to reduce bandwidth pressure.
-

9. Address Coalescing

All GPU loads/stores are gather/scatter.

To regain vector-style unit-stride performance, GPUs detect:

- Adjacent threads accessing adjacent addresses.
- Merge these into a single wide memory transaction.

Programmer responsibility:

- Align data,
 - Ensure threads access adjacent memory,
 - Avoid non-coalesced patterns.
-

10. Fermi Architecture Innovations

The Fermi GPU adds features closer to traditional CPUs:

Key enhancements

1. Dual warp schedulers

- Issue instructions from two warps every two cycles.

2. Much faster double precision

- DP is now 1/2 speed of SP (previously 1/10).

3. Real L1 + L2 caches

- Configurable shared memory / L1 split.

4. 64-bit unified addressing

- Simplifies pointer-based programming.

5. ECC support

- Improves reliability for long-running computation.

6. Fast context switching

- Less than 25 microseconds.

7. Improved atomic operations

- Faster atomic ops via L2 hardware unit.

11. Comparison: Vector Processors vs GPUs

Similarities

- Both perform SIMD operations.
- Both use mask registers for conditional execution.
- Both have lanes executing elements of a vector/warp.

Differences

- Vector systems use:
 - Explicit vector length registers,
 - Deep pipelines,
 - Software-managed masks.
- GPUs use:
 - Hardware-managed masks,
 - Massive multithreading,
 - Stack-based branch control,
 - Address coalescing to recover unit-stride loads.

Key point:

GPUs dedicate silicon to extremely wide SIMD + multithreading rather than scalar processors and caches.

12. Comparison: Multicore CPUs with SIMD vs GPUs

Both have:

- SIMD processing,
- Hardware multithreading,

- 64-bit addressing,
- DP and SP instructions.

But:

Feature	CPU + SIMD	GPU
SIMD lanes	2–4	8–16
SIMD processors	few cores	many SMs
Cache size	large (MBs)	small (KBs–MBs)
Memory coherence	Yes	No (L1s not coherent)
MIMD control flow	Native	Very limited
Demand paging	Supported	Unsupported
Gather-scatter	Limited	Fully supported

13. Summary

GPUs are fundamentally:

Massively multithreaded SIMD machines with hardware mechanisms to mask, branch, diverge, and reconverge control flow — all while sustaining high arithmetic throughput.

The CUDA model hides much of this complexity by presenting:

- A simple “thread-per-element” programming view,
- But optimal performance requires understanding:
 - Warp divergence,
 - Coalesced memory access,
 - Mask behavior,
 - Reconvergence,
 - Thread/block scheduling.

The illusion of independent per-thread execution is only valid when:

- All threads follow the same path,
- Access memory in aligned/coalesced patterns.

Otherwise, the programmer faces significant performance penalties.

If you'd like, I can also produce:

- A **shorter summarized version**,
- A **diagrammatic explanation**,

- A **side-by-side comparison of vector vs GPU behavior**,
- A **custom cheat sheet for GPU branching and divergence**,
- Or an **interview-style explanation** for GPU architecture.

Just tell me which format you want.

I'll go through this section step by step: what loop-level parallelism is, what *loop-carried dependences* are, how we detect them, how we classify/rename dependences, and how we transform recurrences (dot products, reductions) to expose parallelism.

1. What Is Loop-Level Parallelism?

Loops are the main source of data-level and thread-level parallelism in real programs.

- **Loop-level parallelism (LLP)** = running **different iterations of a loop in parallel**.
- It's critical for:
 - **DLP**: vector/SIMD/GPU execution,
 - **TLP**: multithreading,
 - Aggressive **static ILP** styles like VLIW (Appendix H).

Key idea:

A loop is parallel (vectorizable) if **iterations don't depend on results from other iterations**.

Those cross-iteration dependences are called **loop-carried dependences**.

2. Loop-Carried Dependences (Basic Idea)

A **data dependence** arises when:

- One operation **writes** a location,
- A later operation **reads or writes** that same location,
- And there is a potential read-after-write (true dependence), write-after-write (output), or write-after-read (antidependence).

A **loop-carried dependence** specifically involves:

- A write in **iteration i**, and
- A read (or another write) in **iteration j**, with $j > i$.

If such a dependence exists, **iteration j cannot start before iteration i finishes** for that data.

Simple example of a parallel loop

c

```
for (i = 999; i >= 0; i = i - 1)
    x[i] = x[i] + s;
```

- Within an iteration, $x[i] = x[i] + s$; has a dependence:
 - Read $x[i]$, then write $x[i] \rightarrow$ that's fine, it's *within* iteration.
- Between iterations:
 - Iteration i uses only $x[i]$.
 - Iteration $i-1$ uses only $x[i-1]$.
 - No cross-iteration dependence on x .
- There *is* a dependence on the induction variable i itself (i in iteration k depends on i from iteration $k+1$), but compilers know how to handle induction variables—this isn't a data dependence that prevents parallelism.

So the loop is **loop-parallel**: iterations can run in any order or in parallel.

3. More Complex Example: Different Types of Dependences

Consider:

c

```
for (i = 0; i < 100; i = i + 1) {
    A[i+1] = A[i] + C[i]; /* S1 */
    B[i+1] = B[i] + A[i+1]; /* S2 */
}
```

Assume A , B , C are distinct, non-overlapping arrays.

3.1 Dependences present

1. Loop-carried dependence within S1 (and similarly S2):

- $A[i+1]$ depends on $A[i]$ from **previous iteration**.
- That is: to compute $A[i+1]$, we need $A[i]$ from iteration i .
- Similarly, $B[i+1]$ depends on $B[i]$.
- So, S1 and S2 each have loop-carried dependences.

2. Intra-iteration dependence from S1 to S2:

- $B[i+1] = B[i] + A[i+1]$; reads $A[i+1]$ computed by S1 in the **same** iteration.
- That is a **true dependence within the iteration** (not loop-carried).

3.2 Parallelism implications

- The loop-carried dependence of S1 on itself ($A[i+1]$ from $A[i]$) prevents us from freely reordering or overlapping iterations of that loop.

- The S2-on-S1 dependence is *within* each iteration and does **not** prevent **iteration parallelism**, as long as each iteration maintains:
 - S1 $\rightarrow$ S2 order.

Takeaway:

- **Loop-carried dependence on the same statement** (S1(i) depends on S1(i-1)) is usually a barrier to parallelizing the loop.
- **Intra-iteration dependence** can be handled by preserving **statement order** inside iterations (or via chaining, pipelining).

4. Loop-Carried Dependence That *Can* Be Removed

Now consider:

```
c
for (i = 0; i < 100; i = i + 1) {
    A[i] = A[i] + B[i];    /* S1 */
    B[i+1] = C[i] + D[i]; /* S2 */
}
```

4.1 Dependence

- S1(i) uses B[i].
- S2(i-1) writes B[i] (because B[i+1] at iteration i-1 = B[i]).
- So there is a loop-carried **true dependence** from S2(i-1) $\rightarrow$ S1(i).

But notice:

- S1 does **not** produce any value used by S2.
- S2 does **not** use A[i], so there is no dependence back from S1 to S2.
- That means **no cycle**:
 - We have S2(i-1) $\rightarrow$ S1(i), but not S1 $\rightarrow$ S2.

A loop is parallelizable if the dependence graph has **no cycles**. A cycle (e.g., S1(i) depends on S2(i-1) and S2(i) depends on S1(i-1)) typically forces sequential execution.

4.2 Transforming the loop to expose parallelism

We can reorder statements + peel a few iterations to match the partial order and get:

```
c
A[0] = A[0] + B[0];
for (i = 0; i < 99; i = i + 1) {
    B[i+1] = C[i] + D[i];
    A[i+1] = A[i+1] + B[i+1];
}
```

```
B[100] = C[99] + D[99];
```

Now:

- S2 (new) produces **B[i+1]** in the same iteration that S1 (new) uses it.
- So the dependence between the statements is now **within iteration**, not loop-carried.
- The loop body is now parallelizable: different iterations use different **A[i+1]** and **B[i+1]**, and the cross-iteration dependencies are gone (except for those handled by the peeled statements).

5. General Dependence Analysis

The compiler needs to:

1. **Detect** all potential loop-carried dependences.
2. Determine if they truly prevent parallelism.
3. Transform code to reduce/eliminate them where possible.

This is easier at **source level** because:

- You can see loops, array indices, induction variables, etc.
- Machine code has lost much of that structure.

We focus on **data dependences** (true and name dependences). Name dependences (antidependence, output dependence) can often be removed by **renaming**.

6. Dependences Within a Loop: Example

```
c
for (i = 0; i < 100; i = i + 1) {
    A[i] = B[i] + C[i];
    D[i] = A[i] * E[i];
}
```

Here:

- Second use of **A[i]** doesn't need a memory load.
- The compiler knows **A[i]** was just computed, so it can keep it in a register.
- This is only possible if analysis proves:
 - Both references are **definitely** to the same location, and
 - There's no other write to **A[i]** in between.

This is a **must-dependence** (they must refer to the same location), not just a may-dependence.

7. Recurrences (Loop-Carried Self-Dependences)

A **recurrence** is a dependence where a variable's new value depends on its **previous iteration's value**, typically of the same variable.

Example:

```
c
for (i = 1; i < 100; i = i + 1) {
    Y[i] = Y[i-1] + Y[i];
}
```

- $Y[i]$ depends on $Y[i-1]$, directly across iterations.
- This is a classic loop-carried self-dependence.

Recurrences are important because:

- Some architectures (vector processors) have special hardware to execute them efficiently (like special recurrence or prefix-sum support).
- Even in ILP contexts, we can sometimes overlap parts of the recurrence or partially parallelize around it.

8. Formal Dependence Testing for Arrays

Because loops often operate on arrays:

- We have array accesses like $A[a*i + b]$ and $A[c*i + d]$.
- The compiler needs to know whether there exist **two distinct iterations** j and k (within loop bounds) such that:
 - One writes $A[a*j + b]$,
 - Another reads/writes $A[c*k + d]$,
 - And the indices are equal: $a*j + b = c*k + d$.

If such j , k exist (with loop bounds satisfied), a dependence may exist.

Most algorithms assume **affine indices**:

- 1D: $a*i + b$
- Multi-D: each dimension affine in loop indices.

Non-affine, e.g. $x[y[i]]$, is harder (sparse arrays).

8.1 GCD Test (simple test for *no* dependence)

Given indices:

- Store: $a * i + b$

- Load: $c * i + d$

If a loop-carried dependence exists, there must be integers j, k such that:

1. j, k inside loop bounds.
2. $a*j + b = c*k + d$.

A necessary condition: $\text{GCD}(a, c)$ must divide $(d - b)$.

If $\text{GCD}(a, c)$ does **not** divide $(d - b)$, then **no dependence can exist**.

Example:

```
c
for (i = 0; i < 100; i = i + 1) {
    X[2*i+3] = X[2*i] * 5.0;
}
```

- $a = 2, b = 3, c = 2, d = 0$.
- $\text{GCD}(2, 2) = 2$.
- $d - b = 0 - 3 = -3$.
- 2 does **not** divide -3 $\rightarrow$ dependence **impossible**.

So the loop has no loop-carried dependence on x from these accesses.

Note:

- GCD test is **sufficient** to prove independence (no dependence).
- It's not necessary: there are cases where it says "may depend" but in reality there is no dependence (e.g., excluded by loop bounds).

In general, exact dependence testing is **NP-complete**. But:

- In practice, many real-world patterns are simple,
- Compilers use a hierarchy of tests:
 - Cheap but incomplete first (like GCD),
 - More general (and costly) ones later,
 - Some are exact for restricted forms.

9. Classifying Dependences: True, Output, and Antidependences

We classify dependences into:

- **True (flow) dependence:** write $\rightarrow$ read.
- **Output dependence:** write $\rightarrow$ write.
- **Antidependence:** read $\rightarrow$ write.

Only **true dependencies** represent real data flow constraints. The other two (name dependencies) can often be removed via **renaming**.

Example: Many dependencies inside a loop

```
c
for (i = 0; i < 100; i = i + 1) {
    Y[i] = X[i] / c;    /* S1 */
    X[i] = X[i] + c;    /* S2 */
    Z[i] = Y[i] + c;    /* S3 */
    Y[i] = c - Y[i];    /* S4 */
}
```

Dependencies:

1. True dependencies (read-after-write):

- $S1 \rightarrow S3$ on $Y[i]$ ($S3$ reads $Y[i]$ computed by $S1$).
- $S1 \rightarrow S4$ on $Y[i]$.
- These are **within iteration**, not loop-carried.

2. Antidependences (read-before-write):

- $S1 \rightarrow S2$ on $X[i]$: $S1$ reads $X[i]$, $S2$ writes $X[i]$.
- $S3 \rightarrow S4$ on $Y[i]$: $S3$ reads $Y[i]$, $S4$ writes $Y[i]$.

3. Output dependence (write-after-write):

- $S1 \rightarrow S4$ on $Y[i]$: both write $Y[i]$.

These **name dependencies** (antidependence + output dependence) restrict reordering but **do not** represent real data flow.

Eliminating name dependencies via renaming

We rewrite:

```
c
for (i = 0; i < 100; i = i + 1) {
    T[i] = X[i] / c;    /* Y renamed to T to avoid output dependence */
    X1[i] = X[i] + c;   /* X renamed to X1 to remove antidependence */
    Z[i] = T[i] + c;    /* Y renamed to T to remove antidependence */
    Y[i] = c - T[i];
}
```

Changes:

- $Y[i]$ from $S1$ and $S3$ is now $T[i]$.
- X is written into $X1[i]$.
- True data flow remains the same, but:
 - Antidependences and output dependencies are gone.

After loop:

- We treat **x1** as the new **x** (the compiler can just replace all subsequent uses of **x** by **x1**).

Renaming:

- Sometimes only needs **name changes** in the IR/register allocation.
- Sometimes requires real copy instructions (if memory locations must be distinct).

10. Limits of Dependence Analysis

Dependence analysis works best when:

- Accesses are via arrays with **affine indices**,
- The references are within a single **loop nest**,
- There's no heavy pointer aliasing.

Limitations:

- **Pointers, references, pass-by-reference** make analysis hard,
- **Interprocedural** interactions (arrays passed to functions) can be complex,
- Non-affine indexing (**x[y[i]]**, pointer arithmetic) is much harder to analyze.

This is one reason **Fortran** is historically preferred over C/C++ for HPC:

- Fewer aliasing issues,
- Array-based syntax.

Because static analysis is limited, parallel programming models like **OpenMP**, **CUDA**, and others allow programmers to indicate or explicitly write parallel loops.

11. Eliminating Recurrences (e.g., Dot Product)

Consider the classic dot product:

```
c
for (i = 9999; i >= 0; i = i - 1)
    sum = sum + x[i] * y[i];
```

- **sum** is updated each iteration.
- There is a loop-carried true dependence on **sum**.
- This recurrence makes the loop **sequential** as written.

11.1 Transforming into parallel + reduction

Step 1: **Scalar expansion** — break **sum** into a vector of partial results:

```
c
for (i = 9999; i >= 0; i = i - 1)
    sum[i] = x[i] * y[i];
```

This loop has **no cross-iteration dependences** and becomes fully parallel:

- Each iteration writes a distinct `sum[i]`.

Step 2: **Reduce** the vector:

```
c
for (i = 9999; i >= 0; i = i - 1)
    finalsum = finalsum + sum[i];
```

Now:

- The first loop is perfectly parallel (DLP).
- The second loop is a **reduction**:
 - A fold over `sum[i]` with `+`.

11.2 Parallel reduction (multi-processor or vector)

Assume 10 processors:

```
c
for (p = 0; p < 10; p++)
    for (i = 999; i >= 0; i = i - 1)
        finalsum[p] = finalsum[p] + sum[i + 1000 * p];
```

Each processor handles 1000 elements of `sum[]`.

Then a final scalar loop or tree reduction:

- Sums the 10 partial `finalsum[p]` values.

Vector/SIMD hardware uses analogous techniques:

- Partial sums in parallel lanes,
- Then tree-like combining.

11.3 Reduction operator and associativity

We can use *any* associative operator for reduction:

- `+`, `*`, `max`, `min`, etc.

However, **machine arithmetic is not truly associative**:

- Integers → overflow,
- Floating point → rounding errors and limited precision.

So reordering computations (as in parallel reductions) can produce **slightly different results**.

- Usually safe in practice,
 - But compilers often require explicit flags to perform transformations that rely on associativity (e.g., “fast-math” or similar).
-

12. Why All This Matters

Dependence analysis and transformations (like scalar expansion, reduction recognition, renaming, statement reordering, loop transformation) are the **core technology** for:

- Vectorizing compilers,
- Auto-SIMDization,
- Exploiting DLP on GPUs,
- Parallelizing loops for multicore CPUs,
- Enabling aggressive ILP (VLIW, etc.).

Without accurate dependence analysis:

- The compiler either misses large amounts of parallelism,
 - Or generates incorrect code.
-

If you'd like, next we can:

- Work through **specific dependence graphs**,
- Do **manual GCD tests** on your own examples,
- Or rewrite a few loops you give into **vectorizable/parallel form**, step by step.